

ZAALIMA DEVELOPMENT



PYTHON DEVELOPMENT INTERNSHIP

A PROJECT REPORT ON

“Photon : Real-time WebSocket Stock Ticker”

Submitted by

Ayisha Arbin MJ

Under the guidance of

Zaalima Development Team

Acknowledgement

I would like to express my sincere gratitude to the Zaalima Development Team for their continuous guidance, support, and encouragement throughout the first month of my internship.

I am thankful to Zaalima Development for providing me with the opportunity to work on my first Python Development project, which has helped me strengthen my programming skills, understand real-world applications of machine learning, and gain practical experience in building automated systems.

This first project has been an important learning milestone in my internship journey, and I am grateful for the opportunity to apply my knowledge in a practical environment.

Table of contents

Sl. No	Description
--------	-------------

1	Title Page
---	------------

2	Acknowledgement
---	-----------------

Sl. No	Chapter
--------	---------

1	Abstract
---	----------

2	Introduction
---	--------------

3	Problem Statement
---	-------------------

4	Project Objectives
---	--------------------

5	Tools & Technologies Used
---	---------------------------

6	System Architecture / Modules
---	-------------------------------

7	Implementation / Methodology
---	------------------------------

8	Sample Output / Results
---	-------------------------

9	Challenges Faced
---	------------------

10	Future Work / Enhancements
----	----------------------------

11	Conclusion
----	------------

Abstract

Photon is a real-time stock ticker web application developed using Python and WebSocket technology. The main goal of this project is to provide live stock price updates to users without requiring them to refresh the webpage. Traditional stock tracking systems rely on periodic page reloads or API polling, which can cause delays and missed price changes. Photon overcomes this limitation by using WebSockets to push data instantly from the server to the client. The backend is built using FastAPI, which handles WebSocket connections efficiently, while the frontend displays live charts and statistics using HTML, CSS, and JavaScript. This project demonstrates the use of real-time communication in modern web applications.

Introduction

Introduction to project:

In today's fast-paced financial environment, accessing real-time stock market data is crucial for traders, investors, and financial analysts. Even a small delay in price updates can result in missed opportunities or financial loss. Most traditional web applications use request-response mechanisms, where users must refresh the page to get updated data. This approach is inefficient for real-time systems.

The Photon project introduces a real-time stock dashboard that updates automatically using WebSockets. The application provides a seamless experience where data is pushed continuously from the backend server to the frontend interface. This project helps in understanding real-time web technologies and their practical implementation using Python.

Problem Statement

Traditional stock market dashboards rely on manual refresh or periodic polling to fetch updated prices. This approach causes latency, increases server load, and provides a poor user experience. Traders and users require instant price updates to make quick decisions.

The problem addressed in this project is:

Delay in stock price updates

Inefficient use of network resources

Poor real-time user experience

Photon aims to solve these issues by implementing a real-time data streaming system using WebSockets.

Project Objectives

To design a real-time stock price monitoring system

To implement WebSocket-based communication between server and client

To eliminate the need for manual page refresh

To display live stock data visually using charts and statistics

To build a scalable and efficient backend using FastAPI

Tools Us& Technologies programming Language:

Backend Technologies

Python: Core programming language

FastAPI: High-performance web framework

WebSockets: Real-time bidirectional communication

Uvicorn: ASGI server for running FastAPI

Frontend Technologies

HTML: Structure of the web page

CSS: Styling and layout design

JavaScript: Handling WebSocket connections and UI updates

Chart.js: Visual representation of stock data

System Architecture / Modules

The system architecture consists of three main modules:

1. Backend Server

Built using FastAPI

Handles WebSocket connections

Generates or streams stock data

2. WebSocket Communication Layer

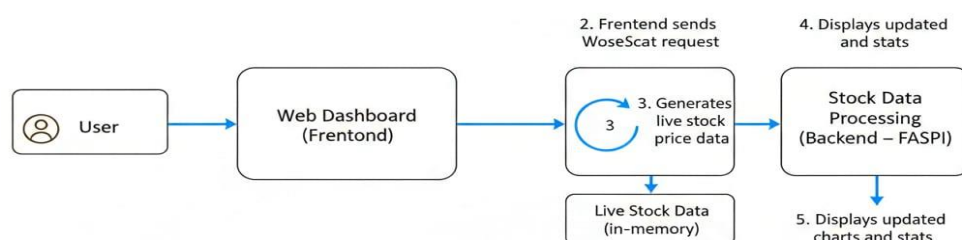
Maintains persistent connections

Sends stock data instantly to all connected clients

3. Frontend Dashboard

Displays live charts and statistics

Updates UI in real time without reload



Implementation / Methodology

The project follows a modular implementation approach:

The user opens the web dashboard in a browser

A WebSocket connection is established with the FastAPI server

The backend generates mock stock price data

Data is sent continuously to the frontend

JavaScript receives and processes the data

Charts and statistics update instantly

This ensures real-time performance and smooth user experience.

Sample Output / Results

The output is a real-time stock dashboard displaying:

Live stock price graph

Last traded price

High and low values

Percentage change

The prices update automatically without any manual refresh. Color indicators show price increase or decrease, making it easy to understand market trends.

Challenges Faced

During the development of the project, the following challenges were faced:

Setting up WebSocket connections correctly

Managing multiple client connections

Handling environment setup (venv issues)

Real-time chart synchronization

Debugging backend and frontend communication

These challenges were resolved through proper configuration and testing.

Future Work / Enhancements

During the development of the project, the following challenges were faced:

Setting up WebSocket connections correctly

Managing multiple client connections

Handling environment setup (venv issues)

Real-time chart synchronization

Debugging backend and frontend communication

These challenges were resolved through proper configuration and testing.

Conclusion

Photon successfully demonstrates a real-time stock price dashboard using WebSockets and FastAPI. The project eliminates the need for page refresh and provides instant data updates, improving efficiency and user experience. It highlights the importance of real-time communication in

modern web applications and serves as a strong foundation for advanced financial systems.